

학습 자료

학습 자료

- Picking 소요 시간 측정 다음과 같이 하세요.

```
Handle WM_LBUTTONDOWN()
{
    // 1) 마우스 화면 좌표 획득
    int screenX = GetMouseScreenX();
    int screenY = GetMouseScreenY();

    // 2) 화면 좌표 -> 월드 좌표로의 픽 레이(Pick Ray) 계산
    Vector pickRay = CalculatePickRay(
        projectionMatrix,
        viewMatrix,
        cameraPosition,
        screenX,
        screenY
    );

    // 3) 퍼포먼스 측정용 카운터 시작
    FScopeCycleCounter pickCounter;

    // 4) 전체 Picking 횟수 누적
    ++TotalPickCount;

    // 5) 모든 오브젝트(프imitives)에 대해 충돌 판정
    for (auto& primitive : primitives)
    {
        bool isHit = primitive.CheckHit(cameraPosition, pickRay);
        // 필요 시 'isHit' 결과를 활용해 추가 로직 처리
    }

    // 6) 퍼포먼스 측정 종료 및 시간 누적
    LastPickTime = pickCounter.Finish();
    TotalPickTime += LastPickTime;
}
```

- RAI (Resource Acquisition Is Initialization)

```

class FWindowsPlatformTime
{
public:
    static double GSecondsPerCycle; // 0
    static bool bInitialized; // false

    static void InitTiming()
    {
        if (!bInitialized)
        {
            bInitialized = true;

            double Frequency = (double)GetFrequency();
            if (Frequency <= 0.0)
            {
                Frequency = 1.0;
            }

            GSecondsPerCycle = 1.0 / Frequency;
        }
    }
    static float GetSecondsPerCycle()
    {
        if (!bInitialized)
        {
            InitTiming();
        }
        return (float)GSecondsPerCycle;
    }
    static uint64 GetFrequency()
    {
        LARGE_INTEGER Frequency;
        QueryPerformanceFrequency(&Frequency);
        return Frequency.QuadPart;
    }
    static double ToMilliseconds(uint64 CycleDiff)
    {
        double Ms = static_cast<double>(CycleDiff)
            * GetSecondsPerCycle()
            * 1000.0;

        return Ms;
    }

    static uint64 Cycles64()
    {
        LARGE_INTEGER CycleCount;
        QueryPerformanceCounter(&CycleCount);
        return (uint64)CycleCount.QuadPart;
    }
};

struct TStatId
{
};

typedef FWindowsPlatformTime FPlatformTime;

class FScopeCycleCounter
{
public:
    FScopeCycleCounter(TStatId StatId)
        : StartCycles(FPlatformTime::Cycles64())
        , UsedStatId(StatId)
    {
    }

    ~FScopeCycleCounter()
    {
        Finish();
    }

    uint64 Finish()
    {
        const uint64 EndCycles = FPlatformTime::Cycles64();
        const uint64 CycleDiff = EndCycles - StartCycles;

        // FThreadStats::AddMessage(UsedStatId, EStatOperation::Add,
        CycleDiff);

        return CycleDiff;
    }
}

```

```
private:
    uint64 StartCycles;
    TStatId UsedStatId;
};
```

- Scene Graph (Management)

[Bounding volume hierarchy - Wikipedia](#)

[K-D Tree](#)

[Octree](#)

- SIMD, Align

[SSE2 - 위키백과, 우리 모두의 백과사전](#)

- Frustum Culling

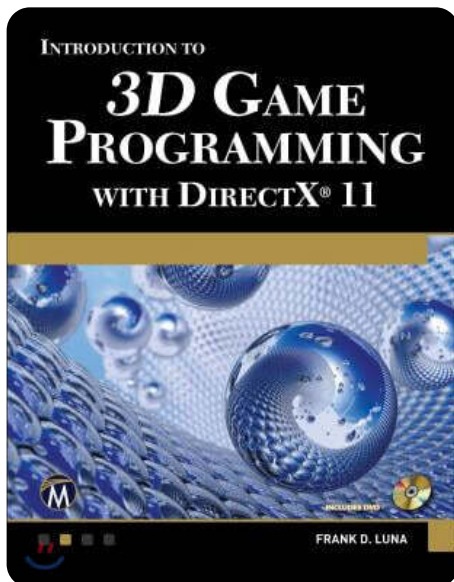
[LearnOpenGL - Frustum Culling](#)

- PIX

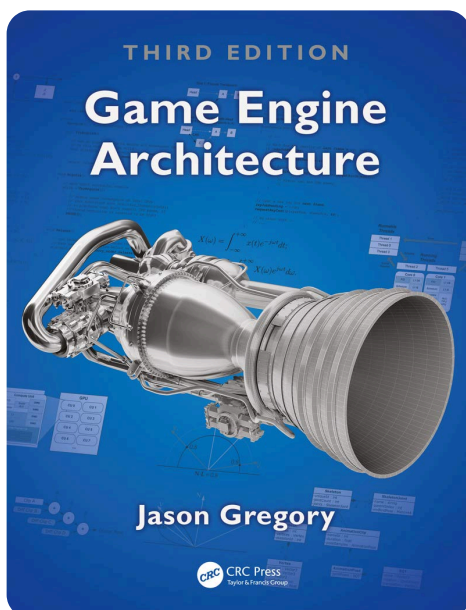
[Download - PIX on Windows](#)



[관련 서적](#)



Chapter 15 - Frustums



GEA 4.10 SIMD

Contact us



Follow us



 Contact site support

강 명호 계정으로 접속하셨습니다. (로그아웃)

[Privacy Policy](#) [Cookie Policy](#)

[Data retention summary](#)

[Get the mobile app](#)

[Get the mobile app](#)